

Namespace F23.StringSimilarity

Classes

[Cosine](#)

[Damerau](#)

Implementation of Damerau-Levenshtein distance with transposition (also sometimes calls unrestricted Damerau-Levenshtein distance). It is the minimum number of operations needed to transform one string into the other, where an operation is defined as an insertion, deletion, or substitution of a single character, or a transposition of two adjacent characters. It does respect triangle inequality, and is thus a metric distance. This is not to be confused with the optimal string alignment distance, which is an extension where no substring can be edited more than once.

[Jaccard](#)

Each input string is converted into a set of n-grams, the Jaccard index is then computed as $|V1 \cap V2| / |V1 \cup V2|$. Like Q-Gram distance, the input strings are first converted into sets of n-grams (sequences of n characters, also called k-shingles), but this time the cardinality of each n-gram is not taken into account. Distance is computed as $1 - \text{cosine similarity}$. Jaccard index is a metric distance.

[JaroWinkler](#)

[Levenshtein](#)

[LongestCommonSubsequence](#)

[MetricLCS](#)

Distance metric based on Longest Common Subsequence, from the notes "An LCS-based string metric" by Daniel Bakkeland.

[NGram](#)

N-Gram Similarity as defined by Kondrak, "N-Gram Similarity and Distance", String Processing and Information Retrieval, Lecture Notes in Computer Science Volume 3772, 2005, pp 115-126.

The algorithm uses affixing with special character '\n' to increase the weight of first characters. The normalization is achieved by dividing the total similarity score the original length of the longest word.

total similarity score the original length of the longest word.

[NormalizedLevenshtein](#)

[OptimalStringAlignment](#)

[QGram](#)

[RatcliffObershelp](#)

Ratcliff/Obershelp pattern recognition

The Ratcliff/Obershelp algorithm computes the similarity of two strings as the doubled number of matching characters divided by the total number of characters in the two strings. Matching characters are those in the longest common subsequence plus, recursively, matching characters in the unmatched region on either side of the longest common subsequence. The Ratcliff/Obershelp distance is computed as $1 - \text{Ratcliff/Obershelp similarity}$.

Author: Ligi <https://github.com/dxpux> (as a patch for fuzzystring) Ported to java from .net by denmase Ported back to .NET by paulirwin to retain compatibility with upstream Java project

[ShingleBased](#)

[SorensenDice](#)

[WeightedLevenshtein](#)

Interfaces

[ICharacterInsDel](#)

As an adjunct to [ICharacterSubstitution](#), this interface allows you to specify the cost of deletion or insertion of a character.

[ICharacterSubstitution](#)

Class Cosine

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class Cosine : ShingleBased, INormalizedStringSimilarity, IStringSimilarity,
    INormalizedStringDistance, IStringDistance
```

Inheritance

[object](#)  ← [ShingleBased](#) ← Cosine

Implements

[INormalizedStringSimilarity](#), [IStringSimilarity](#), [INormalizedStringDistance](#), [IStringDistance](#)

Inherited Members

[ShingleBased.k](#) , [ShingleBased.GetProfile\(string\)](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

Cosine()

Implements Cosine Similarity between strings. The strings are first transformed in vectors of occurrences of k-shingles(sequences of k characters). In this n-dimensional space, the similarity between the two strings is the cosine of their respective vectors.

Default k is 3.

```
public Cosine()
```

Cosine(int)

Implements Cosine Similarity between strings. The strings are first transformed in vectors of occurrences of k-shingles(sequences of k characters). In this n-dimensional space, the similarity between the two strings is the cosine of their respective vectors.

```
public Cosine(int k)
```

Parameters

k [int](#)

Methods

Distance(string, string)

Returns 1.0 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1.0 - the cosine similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(IDictionary<string, int>, IDictionary<string, int>)

```
public double Similarity(IDictionary<string, int> profile1, IDictionary<string, int> profile2)
```

Parameters

profile1 [IDictionary](#) <[string](#), [int](#)>

profile2 [IDictionary](#) <[string](#), [int](#)>

Returns

[double](#)

Similarity(string, string)

Compute the cosine similarity between strings.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The cosine similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Class Damerau

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Implementation of Damerau-Levenshtein distance with transposition (also sometimes calls unrestricted Damerau-Levenshtein distance). It is the minimum number of operations needed to transform one string into the other, where an operation is defined as an insertion, deletion, or substitution of a single character, or a transposition of two adjacent characters. It does respect triangle inequality, and is thus a metric distance. This is not to be confused with the optimal string alignment distance, which is an extension where no substring can be edited more than once.

```
public class Damerau : IMetricStringDistance, IStringDistance,
    IMetricSpanDistance, ISpanDistance
```

Inheritance

[object](#)  ← Damerau

Implements

[IMetricStringDistance](#), [IStringDistance](#), [IMetricSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

Distance(string, string)

Compute the distance between strings: the minimum number of operations needed to transform one string into the other(insertion, deletion, substitution of a single character, or a transposition of two adjacent characters).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The computed distance.

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return the metric distance.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

s2 [ReadOnlySpan](#)<T>

Returns

[double](#)

The metric distance.

Type Parameters

T

Interface ICharacterInsDel

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

As an adjunct to [ICharacterSubstitution](#), this interface allows you to specify the cost of deletion or insertion of a character.

```
public interface ICharacterInsDel
```

Methods

DeletionCost(char)

Computes the deletion cost.

```
double DeletionCost(char c)
```

Parameters

c [char](#) 

The character being deleted.

Returns

[double](#) 

The cost to be allocated to deleting the given character, in the range [0, 1].

InsertionCost(char)

Computes the insertion cost.

```
double InsertionCost(char c)
```

Parameters

c [char](#)

The character being inserted.

Returns

[double](#)

The cost to be allocated to inserting the given character, in the range [0, 1].

Interface ICharacterSubstitution

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public interface ICharacterSubstitution
```

Methods

Cost(char, char)

Indicate the cost of substitution c1 and c2.

```
double Cost(char c1, char c2)
```

Parameters

c1 [char](#)

The first character of the substitution.

c2 [char](#)

The second character of the substitution.

Returns

[double](#)

The cost in the range [0, 1].

Class Jaccard

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Each input string is converted into a set of n-grams, the Jaccard index is then computed as $|V1 \cap V2| / |V1 \cup V2|$. Like Q-Gram distance, the input strings are first converted into sets of n-grams (sequences of n characters, also called k-shingles), but this time the cardinality of each n-gram is not taken into account. Distance is computed as 1 - cosine similarity. Jaccard index is a metric distance.

```
public class Jaccard : ShingleBased, IMetricStringDistance, INormalizedStringDistance,
    IStringDistance, INormalizedStringSimilarity, IStringSimilarity
```

Inheritance

[object](#)  ← [ShingleBased](#) ← Jaccard

Implements

[IMetricStringDistance](#), [INormalizedStringDistance](#), [IStringDistance](#), [INormalizedStringSimilarity](#), [IStringSimilarity](#)

Inherited Members

[ShingleBased.k](#), [ShingleBased.GetProfile\(string\)](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Jaccard()

The strings are first transformed into sets of k-shingles (sequences of k characters), then Jaccard index is computed as $|A \cap B| / |A \cup B|$. The default value of k is 3.

```
public Jaccard()
```

Jaccard(int)

The strings are first transformed into sets of k-shingles (sequences of k characters), then Jaccard index is computed as $|A \cap B| / |A \cup B|$. The default value of k is 3.

```
public Jaccard(int k)
```

Parameters

k [int](#)

Methods

Distance(string, string)

Distance is computed as 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - the Jaccard similarity.

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(string, string)

Compute jaccard index: $|A \cap B| / |A \cup B|$.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#) 

The second string to compare.

Returns

[double](#) 

The Jaccard index in the range [0, 1]

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class JaroWinkler

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class JaroWinkler : INormalizedStringSimilarity, IStringSimilarity,  
    INormalizedStringDistance, IStringDistance, INormalizedSpanSimilarity, ISpanSimilarity,  
    INormalizedSpanDistance, ISpanDistance
```

Inheritance

[object](#) ← JaroWinkler

Implements

[INormalizedStringSimilarity](#), [IStringSimilarity](#), [INormalizedStringDistance](#), [IStringDistance](#),
[INormalizedSpanSimilarity](#), [ISpanSimilarity](#), [INormalizedSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

JaroWinkler()

Creates a new instance with default threshold (0.7)

```
public JaroWinkler()
```

JaroWinkler(double)

Creates a new instance with given threshold to determine when Winkler bonus should be used. Set threshold to a negative value to get the Jaro distance.

```
public JaroWinkler(double threshold)
```

Parameters

threshold [double](#)

Methods

Distance(string, string)

Return 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - similarity

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of distance. Must be ≥ 0 .

This method operates on spans such as byte arrays. Note that, when used on bytes, string encodings that use more than one byte per codepoint (such as UTF-8) are not supported and will most likely return incorrect results.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

s2 [ReadOnlySpan](#)<T>

Returns

[double](#)

The measure of distance between the spans.

Type Parameters

T

Similarity(string, string)

Compute Jaro-Winkler similarity.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The Jaro-Winkler similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Similarity<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of similarity between 2 spans.

```
public double Similarity<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)  <T>

The first span

s2 [ReadOnlySpan](#)  <T>

The second span

Returns

[double](#) 

Similarity (0 means both spans are completely different)

Type Parameters

T

Class Levenshtein

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class Levenshtein : IMetricStringDistance, IStringDistance,
    IMetricSpanDistance, ISpanDistance
```

Inheritance

[object](#) ← Levenshtein

Implements

[IMetricStringDistance](#), [IStringDistance](#), [IMetricSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Equivalent to Distance(s1, s2, Int32.MaxValue).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

Distance(string, string, int)

The Levenshtein distance, or edit distance, between two words is the Minimum number of single-character edits (insertions, deletions or substitutions) required to change one word into the other.

http://en.wikipedia.org/wiki/Levenshtein_distance

It is always at least the difference of the sizes of the two strings. It is at most the length of the longer string. It is zero if and only if the strings are equal. If the strings are the same size, the Hamming distance is an upper bound on the Levenshtein distance. The Levenshtein distance verifies the triangle inequality (the distance between two strings is no greater than the sum Levenshtein distances from a third string).

Implementation uses dynamic programming (Wagner–Fischer algorithm), with only 2 rows of data. The space requirement is thus $O(m)$ and the algorithm runs in $O(mn)$.

```
public double Distance(string s1, string s2, int limit)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

limit [int](#)

The maximum result to compute before stopping. This means that the calculation can terminate early if you only care about strings with a certain similarity. Set this to `Int32.MaxValue` if you want to run the calculation to completion in every case.

Returns

[double](#)

The Levenshtein distance between strings

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return the metric distance.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

s2 [ReadOnlySpan](#)<T>

Returns

[double](#)

The metric distance.

Type Parameters

T

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>, int)

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2, int limit) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

s2 [ReadOnlySpan](#)<T>

limit [int](#)

Returns

[double](#) 

Type Parameters

T

Class LongestCommonSubsequence

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class LongestCommonSubsequence : IStringDistance, ISpanDistance
```

Inheritance

[object](#) ← LongestCommonSubsequence

Implements

[IStringDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Return the LCS distance between strings s1 and s2, computed as $|s1| + |s2| - 2 * |LCS(s1, s2)|$.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The LCS distance between strings s1 and s2, computed as $|s1| + |s2| - 2 * |LCS(s1, s2)|$

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of distance. Must be ≥ 0 .

This method operates on spans such as byte arrays. Note that, when used on bytes, string encodings that use more than one byte per codepoint (such as UTF-8) are not supported and will most likely return incorrect results.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)  <T>

s2 [ReadOnlySpan](#)  <T>

Returns

[double](#) 

The measure of distance between the spans.

Type Parameters

T

Length(string, string)

Return the length of Longest Common Subsequence (LCS) between strings s1 and s2.

```
public int Length(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#) 

The second string to compare.

Returns

[int](#) 

The length of LCS(s1, s2)

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class MetricLCS

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Distance metric based on Longest Common Subsequence, from the notes "An LCS-based string metric" by Daniel Bakkeland.

```
public class MetricLCS : IMetricStringDistance, INormalizedStringDistance, IStringDistance,
    IMetricSpanDistance, ISpanDistance
```

Inheritance

[object](#) ← MetricLCS

Implements

[IMetricStringDistance](#), [INormalizedStringDistance](#), [IStringDistance](#), [IMetricSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Distance metric based on Longest Common Subsequence, computed as $1 - |\text{LCS}(s1, s2)| / \max(|s1|, |s2|)$.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

LCS distance metric

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return the metric distance.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

s2 [ReadOnlySpan](#)<T>

Returns

[double](#)

The metric distance.

Type Parameters

T

Class NGram

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

N-Gram Similarity as defined by Kondrak, "N-Gram Similarity and Distance", String Processing and Information Retrieval, Lecture Notes in Computer Science Volume 3772, 2005, pp 115-126.

The algorithm uses affixing with special character '\n' to increase the weight of first characters. The normalization is achieved by dividing the total similarity score the original length of the longest word.

total similarity score the original length of the longest word.

```
public class NGram : INormalizedStringDistance, IStringDistance
```

Inheritance

[object](#)  ← NGram

Implements

[INormalizedStringDistance](#), [IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

NGram()

```
public NGram()
```

NGram(int)

```
public NGram(int n)
```

Parameters

n [int](#)

Methods

Distance(string, string)

Compute n-gram distance.

```
public double Distance(string s0, string s1)
```

Parameters

s0 [string](#)

The first string to compare.

s1 [string](#)

The second string to compare.

Returns

[double](#)

The computed n-gram distance in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s0 or s1 is null.

Class NormalizedLevenshtein

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class NormalizedLevenshtein : INormalizedStringDistance, IStringDistance,
    INormalizedStringSimilarity, IStringSimilarity, INormalizedSpanDistance, ISpanDistance,
    INormalizedSpanSimilarity, ISpanSimilarity
```

Inheritance

[object](#) ← NormalizedLevenshtein

Implements

[INormalizedStringDistance](#), [IStringDistance](#), [INormalizedStringSimilarity](#), [IStringSimilarity](#),
[INormalizedSpanDistance](#), [ISpanDistance](#), [INormalizedSpanSimilarity](#), [ISpanSimilarity](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Compute distance as Levenshtein(s1, s2) / max(|s1|, |s2|).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The computed distance in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of distance. Must be ≥ 0 .

This method operates on spans such as byte arrays. Note that, when used on bytes, string encodings that use more than one byte per codepoint (such as UTF-8) are not supported and will most likely return incorrect results.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#) <T>

s2 [ReadOnlySpan](#) <T>

Returns

[double](#)

The measure of distance between the spans.

Type Parameters

T

Similarity(string, string)

Return 1 - distance.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - distance

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of similarity between 2 spans.

```
public double Similarity<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first span

s2 [ReadOnlySpan](#)<T>

The second span

Returns

[double](#)

Similarity (0 means both spans are completely different)

Type Parameters

T

Class OptimalStringAlignment

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public sealed class OptimalStringAlignment : IStringDistance, ISpanDistance
```

Inheritance

[object](#) ← OptimalStringAlignment

Implements

[IStringDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Compute the distance between strings: the minimum number of operations needed to transform one string into the other (insertion, deletion, substitution of a single character, or a transposition of two adjacent characters) while no substring is edited more than once.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

the OSA distance

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of distance. Must be ≥ 0 .

This method operates on spans such as byte arrays. Note that, when used on bytes, string encodings that use more than one byte per codepoint (such as UTF-8) are not supported and will most likely return incorrect results.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

s2 [ReadOnlySpan](#)<T>

Returns

[double](#)

The measure of distance between the spans.

Type Parameters

T

Class QGram

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class QGram : ShingleBased, IStringDistance
```

Inheritance

[object](#)  ← [ShingleBased](#) ← QGram

Implements

[IStringDistance](#)

Inherited Members

[ShingleBased.k](#) , [ShingleBased.GetProfile\(string\)](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

QGram()

Q-gram similarity and distance. Defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches", <http://www.sciencedirect.com/science/article/pii/0304397592901434>  The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrence of each k-shingle). Q-gram distance is a lower bound on Levenshtein distance, but can be computed in $O(|A| + |B|)$, where Levenshtein requires $O(|A| \cdot |B|)$ Default k is 3.

```
public QGram()
```

QGram(int)

Q-gram similarity and distance. Defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches", <http://www.sciencedirect.com/science/article/pii/0304397592901434>  The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrences of each k-shingle). Q-gram distance is a lower bound on Levenshtein distance, but can be computed in $O(|A| + |B|)$, where Levenshtein requires $O(|A| \cdot |B|)$

```
public QGram(int k)
```

Parameters

k [int](#)

Methods

Distance(IDictionary<string, int>, IDictionary<string, int>)

Compute QGram distance using precomputed profiles.

```
public double Distance(IDictionary<string, int> profile1, IDictionary<string, int> profile2)
```

Parameters

profile1 [IDictionary](#) <[string](#), [int](#)>

profile2 [IDictionary](#) <[string](#), [int](#)>

Returns

[double](#)

Distance(string, string)

The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrence of each k-shingle).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#) 

The computed Q-gram distance.

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class RatcliffObershelp

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Ratcliff/Obershelp pattern recognition

The Ratcliff/Obershelp algorithm computes the similarity of two strings as the doubled number of matching characters divided by the total number of characters in the two strings. Matching characters are those in the longest common subsequence plus, recursively, matching characters in the unmatched region on either side of the longest common subsequence. The Ratcliff/Obershelp distance is computed as $1 - \text{Ratcliff/Obershelp similarity}$.

Author: Ligi <https://github.com/dxpux> (as a patch for fuzzystring) Ported to java from .net by denmase
Ported back to .NET by paulirwin to retain compatibility with upstream Java project

```
public class RatcliffObershelp : INormalizedStringSimilarity, IStringSimilarity,  
    INormalizedStringDistance, IStringDistance
```

Inheritance

[object](#) ← RatcliffObershelp

Implements

[INormalizedStringSimilarity](#), [IStringSimilarity](#), [INormalizedStringDistance](#), [IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Return $1 - \text{similarity}$.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - similarity

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(string, string)

Compute the Ratcliff-Obershelp similarity between strings.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The RatcliffObershelp similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class ShingleBased

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public abstract class ShingleBased
```

Inheritance

[object](#)  ← ShingleBased

Derived

[Cosine](#), [Jaccard](#), [QGram](#), [SorensenDice](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

ShingleBased()

```
protected ShingleBased()
```

ShingleBased(int)

```
protected ShingleBased(int k)
```

Parameters

k [int](#) 

Exceptions

[ArgumentOutOfRangeException](#) 

If k is less than or equal to 0.

Properties

k

Return k, the length of k-shingles (aka n-grams).

```
protected int k { get; }
```

Property Value

[int](#)

Methods

GetProfile(string)

```
public Dictionary<string, int> GetProfile(string s)
```

Parameters

s [string](#)

Returns

[Dictionary](#) <[string](#), [int](#)>

Class SorensenDice

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class SorensenDice : ShingleBased, INormalizedStringDistance, IStringDistance,
    INormalizedStringSimilarity, IStringSimilarity
```

Inheritance

[object](#)  ← [ShingleBased](#) ← SorensenDice

Implements

[INormalizedStringDistance](#), [IStringDistance](#), [INormalizedStringSimilarity](#), [IStringSimilarity](#)

Inherited Members

[ShingleBased.k](#), [ShingleBased.GetProfile\(string\)](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

SorensenDice()

Sorensen-Dice coefficient, aka Sørensen index, Dice's coefficient or Czekanowski's binary (non-quantitative) index.

The strings are first converted to boolean sets of k-shingles (sequences of k characters), then the similarity is computed as $2 * |A \cap B| / (|A| + |B|)$. Attention: Sorensen-Dice distance (and similarity) does not satisfy triangle inequality. Default k is 3.

```
public SorensenDice()
```

SorensenDice(int)

Sorensen-Dice coefficient, aka Sørensen index, Dice's coefficient or Czekanowski's binary (non-quantitative) index.

The strings are first converted to boolean sets of k-shingles (sequences of k characters), then the similarity is computed as $2 * |A \cap B| / (|A| + |B|)$. Attention: Sorensen-Dice distance (and similarity) does not satisfy triangle inequality.

```
public SorensenDice(int k)
```

Parameters

k [int](#)

Methods

Distance(string, string)

Returns 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1.0 - the computed similarity

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(string, string)

Similarity is computed as $2 * |A \cap B| / (|A| + |B|)$.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#) 

The second string to compare.

Returns

[double](#) 

The computed Sorensen-Dice similarity.

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class WeightedLevenshtein

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class WeightedLevenshtein : IStringDistance
```

Inheritance

[object](#) ← WeightedLevenshtein

Implements

[IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

WeightedLevenshtein(ICharacterSubstitution)

Instantiate with provided character substitution.

```
public WeightedLevenshtein(ICharacterSubstitution characterSubstitution)
```

Parameters

characterSubstitution [ICharacterSubstitution](#)

The strategy to determine character substitution weights.

WeightedLevenshtein(ICharacterSubstitution, ICharacterInsDel)

Instantiate with provided character substitution, insertion, and deletion weights.

```
public WeightedLevenshtein(ICharacterSubstitution characterSubstitution,  
ICharacterInsDel characterInsDel)
```

Parameters

`characterSubstitution` [ICharacterSubstitution](#)

The strategy to determine character substitution weights.

`characterInsDel` [ICharacterInsDel](#)

The strategy to determine character insertion/deletion weights.

Methods

Distance(string, string)

Equivalent to `Distance(s1, s2, Double.MaxValue)`.

```
public double Distance(string s1, string s2)
```

Parameters

`s1` [string](#) 

The first string to compare.

`s2` [string](#) 

The second string to compare.

Returns

[double](#) 

The computed weighted Levenshtein distance.

Distance(string, string, double)

Compute Levenshtein distance using provided weights for substitution.

```
public double Distance(string s1, string s2, double limit)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

limit [double](#)

The maximum result to compute before stopping. This means that the calculation can terminate early if you only care about strings with a certain similarity. Set this to `Double.MaxValue` if you want to run the calculation to completion in every case.

Returns

[double](#)

The computed weighted Levenshtein distance.

Exceptions

[ArgumentNullException](#)

If `s1` or `s2` is null.

Namespace F23.StringSimilarity.Experimental

Classes

[Sift4](#)

Sift4 - a general purpose string distance algorithm inspired by JaroWinkler and Longest Common Subsequence. Original JavaScript algorithm by siderite, java port by Nathan Fischer 2016.

<https://siderite.dev/blog/super-fast-and-accurate-string-distance.html> 

<https://blackdoor.github.io/blog/sift4-java/> 

Class Sift4

Namespace: [F23.StringSimilarity.Experimental](#)

Assembly: F23.StringSimilarity.dll

Sift4 - a general purpose string distance algorithm inspired by JaroWinkler and Longest Common Subsequence. Original JavaScript algorithm by siderite, java port by Nathan Fischer 2016.

<https://siderite.dev/blog/super-fast-and-accurate-string-distance.html>

<https://blackdoor.github.io/blog/sift4-java/>

```
public class Sift4 : IStringDistance
```

Inheritance

[object](#) ← Sift4

Implements

[IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Properties

MaxOffset

Gets or sets the maximum distance to search for character transposition. Compose cost of algorithm is $O(n \cdot \text{MaxOffset})$

```
public int MaxOffset { get; set; }
```

Property Value

[int](#)

Methods

Distance(string, string)

Sift4 - a general purpose string distance algorithm inspired by JaroWinkler and Longest Common Subsequence. Original JavaScript algorithm by siderite, java port by Nathan Fischer 2016.

<https://siderite.dev/blog/super-fast-and-accurate-string-distance.html>

<https://blackdoor.github.io/blog/sift4-java/>

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

s2 [string](#)

Returns

[double](#)

Namespace F23.StringSimilarity.Interfaces

Interfaces

[IMetricSpanDistance](#)

Span distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

[IMetricStringDistance](#)

String distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

[INormalizedSpanDistance](#)

[INormalizedSpanSimilarity](#)

[INormalizedStringDistance](#)

[INormalizedStringSimilarity](#)

[ISpanDistance](#)

[ISpanSimilarity](#)

[IStringDistance](#)

[IStringSimilarity](#)

Interface IMetricSpanDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Span distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

```
public interface IMetricSpanDistance : ISpanDistance
```

Methods

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return the metric distance.

```
double Distance<T>(ReadOnlySpan<T> b1, ReadOnlySpan<T> b2) where T : IEquatable<T>
```

Parameters

b1 [ReadOnlySpan<T>](#)

The first span.

b2 [ReadOnlySpan<T>](#)

The second span.

Returns

[double](#)

The metric distance.

Type Parameters

Interface IMetricStringDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

String distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

```
public interface IMetricStringDistance : IStringDistance
```

Methods

Distance(string, string)

Compute and return the metric distance.

```
double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

s2 [string](#) 

Returns

[double](#) 

Interface INormalizedSpanDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface INormalizedSpanDistance : ISpanDistance
```

Inherited Members

[ISpanDistance.Distance<T>\(ReadOnlySpan<T>, ReadOnlySpan<T>\)](#).

Interface INormalizedSpanSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface INormalizedSpanSimilarity : ISpanSimilarity
```

Inherited Members

[ISpanSimilarity.Similarity<T>\(ReadOnlySpan<T>, ReadOnlySpan<T>\)](#).

Interface INormalizedStringDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface INormalizedStringDistance : IStringDistance
```

Inherited Members

[IStringDistance.Distance\(string, string\)](#).

Interface INormalizedStringSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface INormalizedStringSimilarity : IStringSimilarity
```

Inherited Members

[IStringSimilarity.Similarity\(string, string\)](#)

Interface ISpanDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface ISpanDistance
```

Methods

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of distance. Must be ≥ 0 .

This method operates on spans such as byte arrays. Note that, when used on bytes, string encodings that use more than one byte per codepoint (such as UTF-8) are not supported and will most likely return incorrect results.

```
double Distance<T>(ReadOnlySpan<T> b1, ReadOnlySpan<T> b2) where T : IEquatable<T>
```

Parameters

b1 [ReadOnlySpan<T>](#)

The first span.

b2 [ReadOnlySpan<T>](#)

The second span.

Returns

[double](#)

The measure of distance between the spans.

Type Parameters

T

Interface ISpanSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface ISpanSimilarity
```

Methods

Similarity<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of similarity between 2 spans.

```
double Similarity<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first span

s2 [ReadOnlySpan](#)<T>

The second span

Returns

[double](#)

Similarity (0 means both spans are completely different)

Type Parameters

T

Interface IStringDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface IStringDistance
```

Methods

Distance(string, string)

Compute and return a measure of distance. Must be ≥ 0 .

```
double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

s2 [string](#) 

Returns

[double](#) 

Interface IStringSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

```
public interface IStringSimilarity
```

Methods

Similarity(string, string)

Compute and return a measure of similarity between 2 strings.

```
double Similarity(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string

s2 [string](#) 

The second string

Returns

[double](#) 

Similarity (0 means both strings are completely different)